

Apresentação do Projeto Final de
Fundamentos de Banco de Dados

UniCarona

Professor: Francisco Victor da Silva Pinheiro

Alunos:

Francisca Arianne dos Santos da Silva – 554930

Gabriel Braga Martins – 493659

Pablo Brandão Passos – 539730

Raul Camurça Rabelo de Almeida – 596042

Quixadá-CE
03 de Julho de 2026

O Cenário e a Proposta do UniCarona



- **Problema:** Dificuldade de deslocamento diário e custo de transporte para estudantes universitários.
- **Solução:** Uma plataforma centralizada de caronas solidárias focada na comunidade acadêmica.
- **Objetivo do Sistema:** Criar uma rede de caronas colaborativa, segura e restrita à comunidade acadêmica, otimizando a rotina de deslocamento dos estudantes.

Diagrama ER

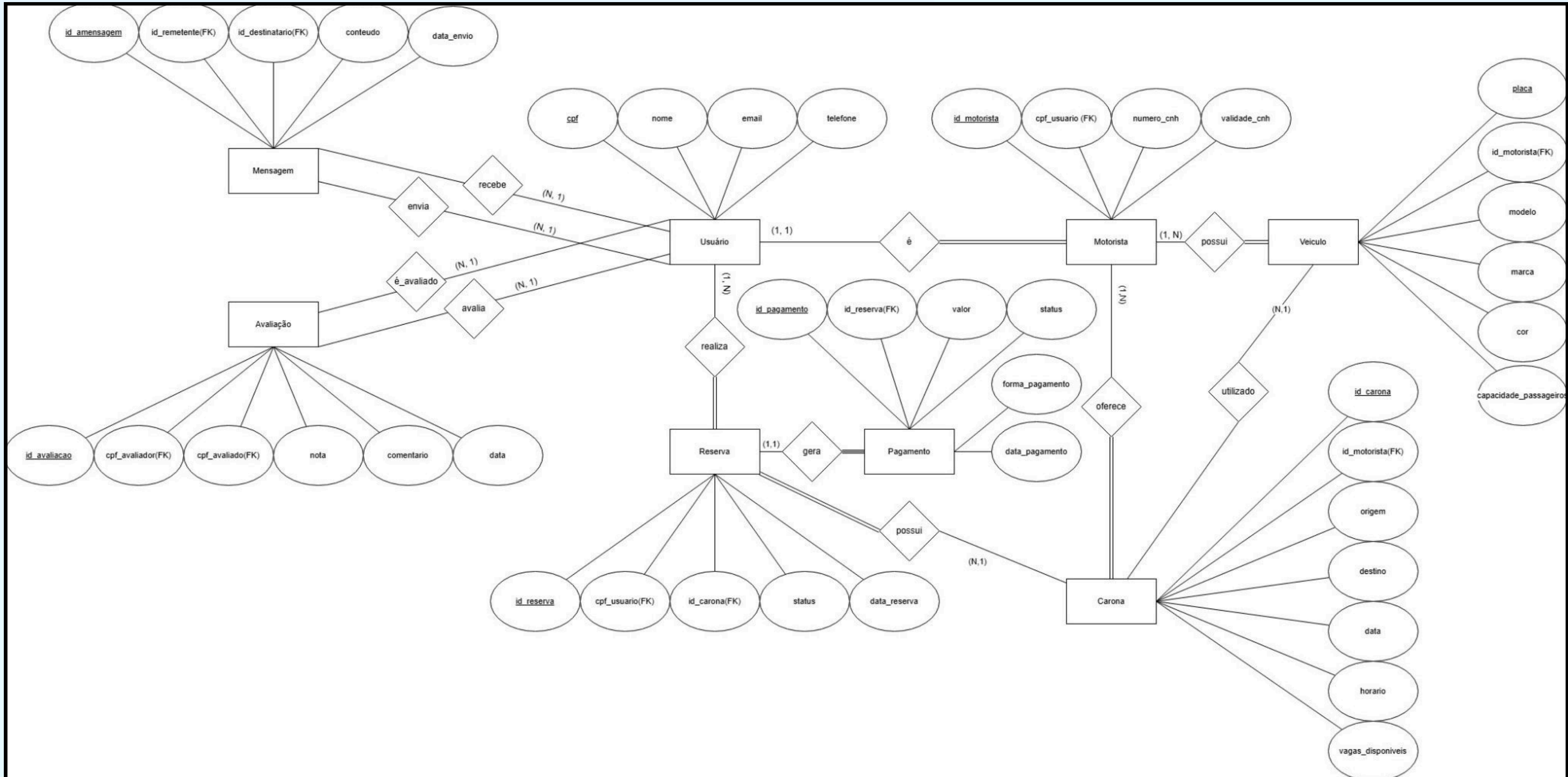
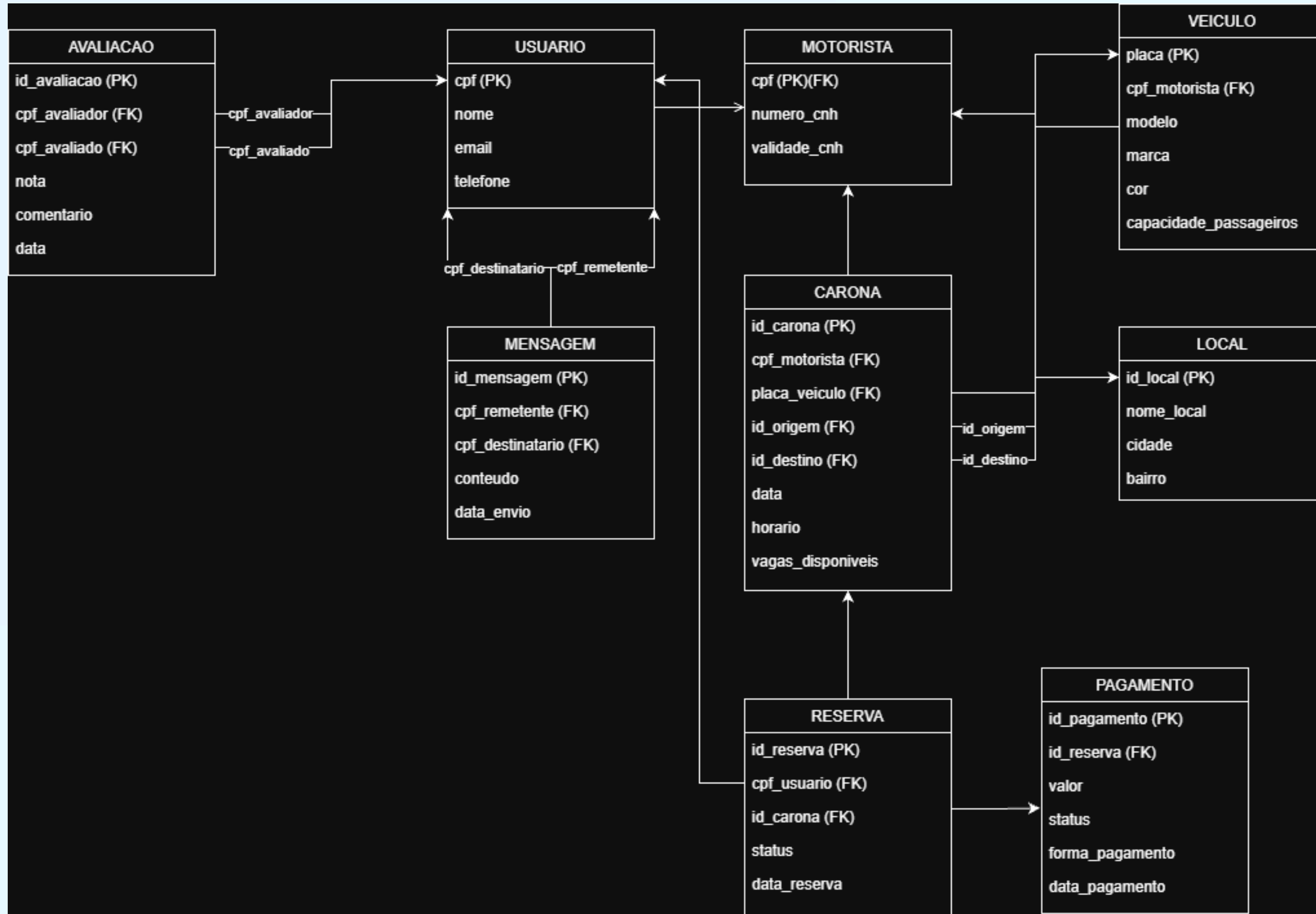


Diagrama Relacional



Consulta 2

- Um relatório complexo que utiliza uma junção múltipla de seis tabelas (Multi-JOIN) para consolidar todas as variáveis de uma viagem em uma única linha (motorista, veículo, pontos geográficos de origem e destino mapeados).
- Além da projeção, a consulta executa uma expressão aritmética dinâmica no SELECT, subtraindo as vagas disponíveis da capacidade total do veículo para retornar a quantidade exata de passageiros alocados por viagem.

```
1  SELECT C.id_carona, U.nome AS motorista, V.modelo AS veiculo,
2         L1.nome_local AS origem, L2.nome_local AS destino,
3         (V.capacidade_passageiros - C.vagas_disponiveis) AS passageiros_alocados
4  FROM CARONA C
5  JOIN MOTORISTA M ON C.cpf_motorista = M.cpf
6  JOIN USUARIO U ON M.cpf = U.cpf
7  JOIN VEICULO V ON C.placa_veiculo = V.placa
8  JOIN LOCAL L1 ON C.id_origem = L1.id_local
9  JOIN LOCAL L2 ON C.id_destino = L2.id_local;
```


Consulta 2 - pgAdmin

Data Output Messages Notifications						
☰ 📄 ▼ 📋 ▼ 🗑️ 🗑️ ⬇️ 📈 SQL						
	Id_carona integer 🔒	motorista character varying (100) 🔒	veiculo character varying (50) 🔒	origem character varying (100) 🔒	destino character varying (100) 🔒	passageiros_alocados integer 🔒
1	32	Aline Barros	Corolla	Centro	UFC Quixadá	1
2	33	Aline Barros	Corolla	UFC Quixadá	IFCE	2
3	34	Ricardo Gomes	HB20	UFC Quixadá	Centro	0
4	35	Ricardo Gomes	HB20	Centro	UFC Quixadá	0
5	36	João Almeida	Onix	Rodoviária	IFCE	1
6	1	Thiago Cunha	Renegade	UFC Quixadá	Bloco de Direito - UFC	0
7	2	Tatiana Fontes	Tracker	Rodoviária	Bloco de Engenharia - UFC	0
8	3	Vitor Rezende	Equinox	Shopping	Restaurante Universitário - U...	0
9	4	Vanessa Nunes	Spin	Centro	Biblioteca Central - UFC	2
10	5	Yago Martins	Cobalt	IFCE	Auditório Principal - UFC	0
11	6	Yasmin Toledo	S10	Bloco de Direito - UFC	Praça da Leão	0
12	7	Arthur Pegoraro	Ranger	Bloco de Engenharia - UFC	Terminal Rodoviário	0
13	8	Beatriz Arantes	Hilux	Restaurante Universitário - U...	Praça da Estação	0
14	9	Caio Gaspar	Amarok	Biblioteca Central - UFC	Alto São Francisco	0
15	10	Diana Siqueira	T-Cross	Auditório Principal - UFC	Igreja Campo Novo	0
16	11	Erick Farias	Nivus	Praça da Leão	Posto Carrascal	0
17	12	Flávia Viana	Taos	Terminal Rodoviário	Avenida Cedro	0
18	13	Guilherme Macedo	Tiguan	Praça da Estação	Ponto do Combate	0
19	14	Ingrid Arruda	Jetta	Alto São Francisco	Rotatória Curicaca	0
20	15	Júlio Medeiros	Golf	Igreja Campo Novo	Estação Baviera	0
21	16	Karina Paiva	Vovane	Posto Carrascal	Hospital Eudásio Barroso	0
Total rows: 135		Query complete 00:00:00.131				

Consulta 3

- Esta consulta implementa uma estrutura avançada de agregação e subconsulta aninhada. O escopo interno agrupa as reservas e calcula a soma dos pagamentos aprovados por carona.
- A cláusula HAVING em tempo de execução atua aplicando um pós-filtro que compara o faturamento de cada grupo individual contra a média aritmética global de repasses de todo o sistema, isolando apenas as viagens de alta rentabilidade.

```
1  SELECT C.id_carona, C.data, C.horario, SUM(P.valor) AS faturamento_carona
2  FROM CARONA C
3  JOIN RESERVA R ON C.id_carona = R.id_carona
4  JOIN PAGAMENTO P ON R.id_reserva = P.id_reserva
5  WHERE P.status = 'Aprovado'
6  GROUP BY C.id_carona, C.data, C.horario
7  HAVING SUM(P.valor) > (
8      SELECT AVG(faturamento_total)
9      FROM (
10         SELECT R2.id_carona, SUM(P2.valor) AS faturamento_total
11         FROM RESERVA R2
12         JOIN PAGAMENTO P2 ON R2.id_reserva = P2.id_reserva
13         WHERE P2.status = 'Aprovado'
14         GROUP BY R2.id_carona
15     ) AS medias_gerais
16 );
```

Consulta 3 - pgAdmin



Data Output Messages Notifications				
☰+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL				
	id_carona [PK] integer 	data date 	horario time without time zone 	faturamento_carona numeric 
1	10	2026-06-...	17:00:00	18.00
2	6	2026-06-...	13:00:00	12.50
3	31	2026-07-...	06:00:00	87.50
4	7	2026-06-...	14:00:00	15.00
5	38	2026-07-...	22:00:00	14.00
6	37	2026-07-...	21:00:00	15.50
7	33	2026-07-...	06:30:00	22.00
Total rows: 7		Query complete 00:00:00.769		

Consulta 8

- Esta consulta utiliza uma estratégia de otimização de subconsulta de exclusão por meio da cláusula NOT EXISTS.
- O banco mapeia todos os veículos cadastrados na plataforma e remove do resultado aqueles cujas placas aparecem registradas em algum histórico de viagem na tabela CARONA, retornando a lista de carros ociosos.

```
1  SELECT V.placa, V.modelo, V.marca
2  FROM VEICULO V
3  WHERE NOT EXISTS (
4      SELECT 1
5      FROM CARONA C
6      WHERE C.placa_veiculo = V.placa
7  );
```



Consulta 8 - pgadmin



Data Output Messages Notifications			
☰+  ▼  ▼     SQL			
	placa [PK] character varying (7) 	modelo character varying (50) 	marca character varying (50) 
1	OXX1065	Weekend	Fiat
2	OXX1079	C4	Citroen
3	OXX1126	Civic	Honda
4	OXX1127	Sandero	Renault
Total rows: 4		Query complete 00:00:00.129	

Visão 1 - vw_detalhes_caronas

- Consolida os dados operacionais de todas as caronas, trazendo os nomes do motorista, veículo, placa, origem, destino, data, horário e vagas disponíveis.
- Evita que a aplicação tenha que fazer 5 JOINS toda vez que listar viagens (por exemplo).



```
1  CREATE OR REPLACE VIEW vw_detalhes_caronas AS
2  SELECT C.id_carona,
3         U.nome AS motorista,
4         V.modelo AS veiculo,
5         V.placa,
6         L1.nome_local AS origem,
7         L2.nome_local AS destino,
8         C.data,
9         C.horario,
10        C.vagas_disponiveis
11 FROM CARONA C
12 JOIN MOTORISTA M ON C.cpf_motorista = M.cpf
13 JOIN USUARIO U ON M.cpf = U.cpf
14 JOIN VEICULO V ON C.placa_veiculo = V.placa
15 JOIN LOCAL L1 ON C.id_origem = L1.id_local
16 JOIN LOCAL L2 ON C.id_destino = L2.id_local;
```

[illegible]

Visão 2 - vw_ranking_motoristas

- Agrega as notas da tabela de avaliações para gerar um ranking de reputação dos motoristas com média arredondada, filtrando apenas quem tem boa atividade na plataforma.



```
1 CREATE OR REPLACE VIEW vw_ranking_motoristas AS
2 SELECT U.nome AS motorista,
3        ROUND(AVG(A.nota)::numeric, 2) AS media_reputacao,
4        COUNT(A.id_avaliacao) AS total_avaliacoes_recebidas
5 FROM AVALIACAO A
6 JOIN MOTORISTA M ON A.cpf_avaliado = M.cpf
7 JOIN USUARIO U ON M.cpf = U.cpf
8 GROUP BY U.cpf, U.nome
9 ORDER BY media_reputacao DESC;
```


Visão 2 - vw_ranking_motoristas

Data Output Messages Notifications			
≡+ ▼ ▼ SQL			
	motorista character varying (100)	media_reputacao numeric	total_avaliacoes_recebidas bigint
1	Aline Barros	4.94	32
2	João Almeida	4.06	31
Total rows: 2 Query complete 00:00:00.162			

Segurança e Controle de Acesso (DCL)

```
1  -- Criando os usuários
2  CREATE USER unicarona_admin WITH PASSWORD 'admin_unicarona_2026';
3  CREATE USER unicarona_leitura WITH PASSWORD 'leitura_unicarona_2026';
4
5  -- 1. Permissões do Usuário Admin (Privilégios totais de leitura e escrita)
6  GRANT ALL PRIVILEGES ON DATABASE unicarona TO unicarona_admin;
7  GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO unicarona_admin;
8  GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO unicarona_admin;
9
10 -- 2. Permissões do Usuário Somente Leitura (Apenas SELECT)
11 -- Revogando qualquer direito de escrita
12 REVOKE INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public FROM unicarona_leitura;
13
14 -- Concedendo estritamente o acesso de leitura nas tabelas e visões
15 GRANT CONNECT ON DATABASE unicarona TO unicarona_leitura;
16 GRANT USAGE ON SCHEMA public TO unicarona_leitura;
17 GRANT SELECT ON ALL TABLES IN SCHEMA public TO unicarona_leitura;
18 GRANT SELECT ON ALL SEQUENCES IN SCHEMA public TO unicarona_leitura;
```

Implementação do CRUD



O foco da aplicação foi implementar o CRUD (Create, Read, Update, Delete) central do nosso sistema: o gerenciamento da tabela CARONA. As seguintes funcionalidades (rotas) foram disponibilizadas e testadas via interface do Swagger UI:

- **Create (*POST /caronas/*)**: Permite inserir uma nova oferta de carona no sistema, vinculando motorista, veículo, locais de origem/destino e vagas.
- **Read (*GET /caronas/* e */caronas/{id}*)**: Permite listar todas as caronas disponíveis ou buscar os detalhes de uma viagem específica pelo seu identificador.
- **Update (*PUT /caronas/{id}*)**: Permite atualizar informações de uma carona existente (ex: diminuir a quantidade de vagas disponíveis após uma reserva).
- **Delete (*DELETE /caronas/{id}*)**: Permite cancelar/remover uma carona do banco de dados.

O funcionamento do CRUD no Swagger pode ser observado a partir do repositório no link abaixo:

https://github.com/PabloBr4ndao/Fundamentos_BancoDeDados/tree/main/Projeto_Unicarona/printscreens_app

FIM!

OBRIGADO PELA ATENÇÃO!



UNIVERSIDADE
FEDERAL DO CEARÁ